CPSC 250 - Programming for Data Manipulation
Test 3

Name:_____

Instructions: Answer all questions on this test paper. You may not use any devices. Write clearly and show your work where needed.

Topics Covered: Derived classes, constructors and `super()`, inheritance, polymorphism, lists of mixed object types, recursion, binary search, and algorithm efficiency.

Total Points: 80

Part 1 - Multiple Choice (10 points)

Circle the best answer. Each question is worth 2 points.

1. What is the main purpose of inheritance in object-oriented programming?

   a) To make every variable private
   b) To allow one class to reuse and extend another class
   c) To prevent methods from being overridden
   d) To make recursive functions faster

2. In a derived class constructor, why do we usually call `super().__init__(...)`?

   a) To initialize the base-class portion of the object
   b) To delete the base-class variables
   c) To force Python to use polymorphism
   d) To make the object printable

3. Which statement best describes polymorphism?

   a) Many classes can respond to the same method call in their own way
   b) A class can only have one constructor
   c) A function can only return one type of value
   d) A list can only store objects of one class

4. Consider the following code.

```
class Animal:
    def speak(self):
        return "..."

class Cat(Animal):
    def speak(self):
        return "meow"

a = Cat()
print(a.speak())
```

What is printed?

a) `...`
b) `meow`
c) `Cat`
d) An error occurs because `speak()` is defined twice

5. Why is recursive Fibonacci much slower than an iterative Fibonacci algorithm?

a) It uses a loop incorrectly
b) It repeatedly recomputes the same smaller Fibonacci values
c) It cannot have a base case
d) It always searches a sorted list

Part 2 - Find the Errors (15 points)

Each question contains errors related to this week's topics. Identify the errors and briefly explain how to fix them.

6. Derived class constructor

```
class Person:
    def __init__(self, name):
        self.name = name

class Student(Person):
    def __init__(self, name, student_id):
        self.student_id = student_id

    def print_info(self):
        print(self.name, self.student_id)

s = Student("Alice", "A123")
s.print_info()
```

7. Method overriding and object creation

```
class Instrument:
    def play(self):
        return "sound"

class Guitar(Instrument):
    def play():
        return "strum"

g = Guitar
print(g.play())
```

8. Recursive binary search

The function below is supposed to return `True` if `target` is in the sorted list and `False` otherwise.

```
def binary_search(values, target, low, high):
    mid = (low + high) // 2

    if values[mid] == target:
        return True

    elif target < values[mid]:
        return binary_search(values, target, low, mid)

    else:
        return binary_search(values, target, mid, high)
```

Part 3 - Code Tracing and Algorithm Reasoning (20 points)

9. Polymorphism tracing (10 points)

What is printed by the following program?

```
class Item:
    def __init__(self, name):
        self.name = name

    def label(self):
        return "Item: " + self.name

class Food(Item):
    def __init__(self, name, calories):
        super().__init__(name)
        self.calories = calories

    def label(self):
        return "Food: " + self.name + " (" + str(self.calories) + ")"

class Tool(Item):
    def label(self):
        return "Tool: " + self.name

things = [
    Item("box"),
    Food("apple", 95),
    Tool("hammer")
]

for thing in things:
    print(thing.label())
```

10. Recursive function tracing (10 points)

Consider the following function.

```
def mystery(n):
    if n <= 1:
        return 1
    else:
        return n + mystery(n - 2)
```

a) What is the value of `mystery(6)`? Show the sequence of calls or partial sums.

b) What is the base case?

c) Does this function behave more like Fibonacci recursion or simple countdown recursion? Briefly explain.

Part 4 - Code Writing (25 points)

Answer each question clearly. Use correct syntax and indentation.

11. Inheritance and `super()` (12 points)

Write a base class called `MediaItem` with attributes `title` and `year`. Then write a derived class called `Song` that adds an attribute `artist`.

Your classes should include:

- constructors for both classes,
- a call to `super().__init__()` inside the `Song` constructor,
- a `__str__()` method for `Song` that returns a string such as:

```
"Fast Car by Tracy Chapman (1988)"
```

12. Mixed list and polymorphism (8 points)

Using the classes below, write code that creates a list containing two `Plant` objects and two `Flower` objects. Then use a loop to call `print_info()` for every object in the list.

```
class Plant:
    def __init__(self, name, cost):
        self.name = name
        self.cost = cost

    def print_info(self):
        print("Plant:", self.name, self.cost)

class Flower(Plant):
    def __init__(self, name, cost, color):
        super().__init__(name, cost)
        self.color = color

    def print_info(self):
        print("Flower:", self.name, self.cost, self.color)
```

13. Iterative Fibonacci (5 points)

Write a function called `fib_iter(n)` that returns the nth Fibonacci number using a loop, not recursion. Assume:

```
fib_iter(0) returns 0
fib_iter(1) returns 1
```

Part 5 - Short Answer (10 points)

14. Recursion and end conditions (5 points)

Explain why every recursive function needs an end condition. What happens if the end condition is missing or never reached?

15. Binary search efficiency (5 points)

Binary search only works correctly under certain conditions. What condition must be true about the data, and why is binary search more efficient than checking every value one at a time?

End of Test